

Lecture 19

Generative AI / Physical AI

VLA Lab: OpenVLA Fine-Tuning

I050-4-6338-01 · Spring 2026

Instructor: Prof. Yong-Hoon Choi

Contributors



Jungwoo Lee



Chaewon Park



Jeonghyeok Park

Use GitHub Repository

- https://github.com/KWU-FAIR-LAB/Raccoonbot_Openvla

Use Hugging Face (already fine-tuned model)

- <https://huggingface.co/fair-lab/openvla-7b-finetuned-raccoonbot/tree/main>

RaccoonBot + OpenVLA Installation

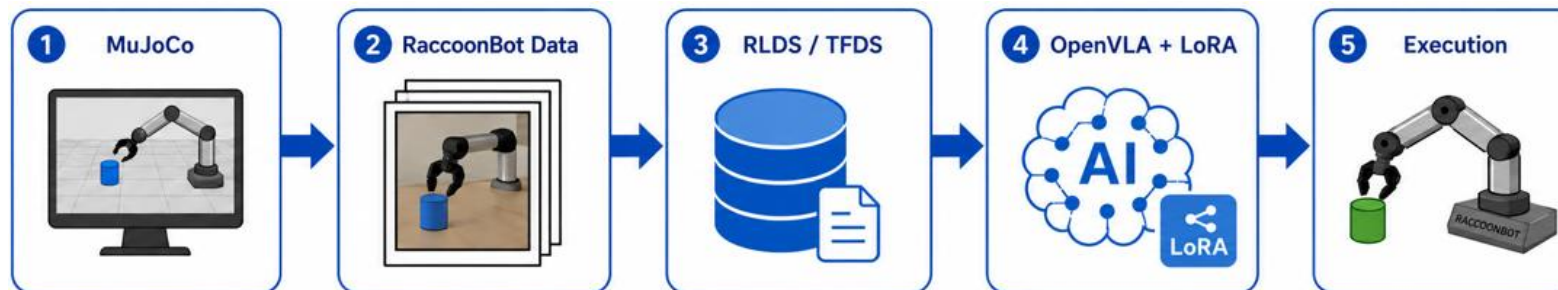
Adapting OpenVLA to a 4-DOF Robot

❖ What We Will Do

- Simulate RaccoonBot in **MuJoCo**.
- Generate robot-specific grasp demonstrations.
- Convert data into **RLDS** / TFDS format.
- Fine-tune OpenVLA with **LoRA** adapters.

❖ Main Points

- MuJoCo creates training demonstrations.
- RLDS / TFDS makes the data OpenVLA-compatible.
- LoRA enables efficient adaptation without full fine-tuning.
- A 7D VLA action format must be adapted to a **4-DOF robot**.



❖ Where Each Part Is Implemented

- Raccoonbot_Openvla/
 - ✓ Mujoco/ → simulation & dataset generation
 - ✓ openvla/ → fine-tuning & inference server
 - ✓ dlimp_openvla/ → data processing utilities

❖ Key Files to Follow

- raccoon_grasp_multicolor_scene_dataset.py → generate MuJoCo demonstrations
- convert_raw_to_openvla_rlds_intermediate.py → convert raw data to RLDS-style format
- raccoon_pick_place_dataset_builder.py → build TFDS dataset
- openvla_server.py → run OpenVLA inference server
- vla-scripts/finetune.py → LoRA fine-tuning (OpenVLA official)

Target Task

❖ Task

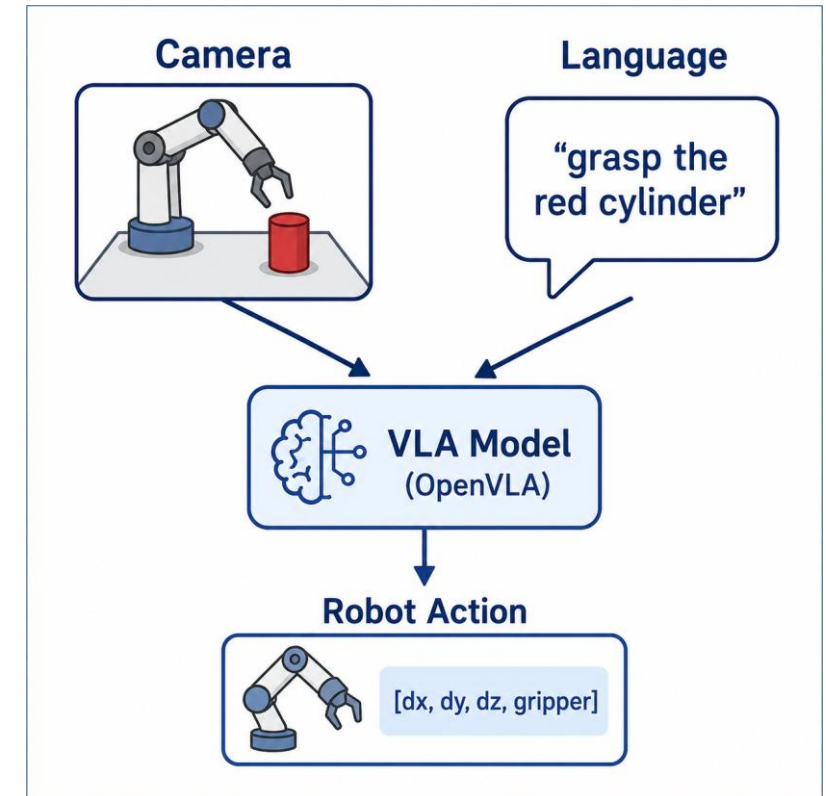
- Color-conditioned grasping

❖ Language Instructions

- "grasp the red cylinder"
- "grasp the blue cylinder"
- "grasp the green cylinder"
- "grasp the yellow cylinder"

❖ Main Points

- The MuJoCo scene contains four colored cylinders.
- The instruction specifies which object to grasp.
- RaccoonBot moves toward the target cylinder.
- The gripper closes to complete the grasp.



Step 0: Repository Setup

❖ Goal

- Prepare the repository and OpenVLA environment.

❖ Commands

- `git clone https://github.com/KWU-FAIR-LAB/Raccoonbot_Openvla.git`
- `apt install ...` # OpenGL / EGL packages for MuJoCo rendering
- `cd Raccoonbot_Openvla/openvla`
- `pip install .` # pip install /data/code/Raccoonbot_Openvla/openvla

❖ Main Points

- Clone the GitHub repository first.
- Install required system packages if needed.
- Install the local OpenVLA package.
- Then proceed to dataset generation or inference.

In Notebook:
Use %cd for directory changes.
Use ! to run shell commands.

```
apt update
apt install -y \
  libegl1 \
  libgl1 \
  libglvnd0 \
  libglx0 \
  libopengl0 \
  libgles2 \
  libegl1-mesa \
  libegl1-mesa-dev \
  mesa-utils

cd Raccoonbot_Openvla/openvla
pip install .
```

Step 0: Repository Setup

The screenshot shows a JupyterLab environment. On the left, the file explorer displays the directory structure: `/code/Raccoonbot_Openvla/`. It lists files: `dlimp_open...`, `Mujoco`, `openvla`, and `README.md`, all modified 6 minutes ago. The main area shows a Jupyter notebook named `raccoon_lab1.ipynb` with the following code cells:

```
[1]: !pwd

/data/code

[2]: !git clone https://github.com/KWU-FAIR-LAB/Raccoonbot_Openvla.git

Cloning into 'Raccoonbot_Openvla'...
remote: Enumerating objects: 304, done.
remote: Counting objects: 100% (304/304), done.
remote: Compressing objects: 100% (255/255), done.255
remote: Total 304 (delta 57), reused 242 (delta 27), pack-reused 0 (from 0)
Receiving objects: 100% (304/304), 6.53 MiB | 1.79 MiB/s, done.
Resolving deltas: 100% (57/57), done.

[3]: !ls -alF

total 344
drwxr-xr-x  5 root root  4096 May 24 06:59 ./
drwxr-xr-x 14 root root  4096 Jun  6 2025 ../
drwxr-xr-x  2 root root  4096 May 24 06:59 .ipynb_checkpoints/
drwxr-xr-x  3 root root  4096 May 24 06:59 AIP/
-rw-r--r--  1 root root 327567 Apr 22 2024 FinGPT_Training_LoRA_with_ChatGLM2_6B_for_Beginners_v2-2_fixed.ipynb
drwxr-xr-x  6 root root  4096 May 24 06:59 Raccoonbot_Openvla/
-rw-r--r--  1 root root    72 May 24 06:59 Untitled.ipynb

[9]: %cd /data/code/Raccoonbot_Openvla

/data/code/Raccoonbot_Openvla
```


Step 0: Repository Setup

❖ Fixing a Local Path Issue

- Create a symbolic link from the expected path to the actual path.
 - ✓ `ln -s /data/code/Raccoonbot_Openvla /data/Raccoonbot_Openvla`
 - ✓ `cd /data/code/Raccoonbot_Openvla/openvla`
 - ✓ `pip install .`

```
2.2/2.2 GB 1.6 MB/s eta 0:00:0000:0100:12m
Processing /data/Raccoonbot_Openvla/dlimp_openvla
ERROR: Could not install packages due to an OSError: [Errno 2] No such file or directory: '/data/Raccoonbot_Openvla/dlimp_openvla'
```

```
WARNING: You are using pip version 22.0.4; however, version 25.0.1 is available.
You should consider upgrading via the '/usr/bin/python3.8 -m pip install --upgrade pip' command.
```

```
[16]: !ln -s /data/code/Raccoonbot_Openvla /data/Raccoonbot_Openvla
```

```
[17]: %cd /data/code/Raccoonbot_Openvla/openvla
/data/code/Raccoonbot_Openvla/openvla
```

```
[*]: !pip install .
```

```
Processing /data/code/Raccoonbot_Openvla/openvla
```

MuJoCo Demonstration Data

MuJoCo Demonstration Generation

❖ Goal

- Generate RaccoonBot-specific grasp demonstrations.

❖ Main Script

- `raccoon_grasp_multicolor_scene_dataset.py`

❖ What the Script Does

- Randomize colored cylinder positions.
- Create a language instruction.
- Execute a rule-based grasp trajectory.
- Save images, states, actions, and metadata

❖ Output

- Episode folders with image frames and meta.json.

/ ... / Raccoonbot_Openvla / Mujoco /	
Name	Last Modified
assets	an hour ago
raccoon_dataset	an hour ago
rlds_dataset_builder	an hour ago
Raccoon_colored_c...	an hour ago
raccoon_env.py	an hour ago
raccoon_grasp_mul...	an hour ago
RaccoonBot_S.xml	an hour ago

Raw Demonstration Dataset

❖ Episode-Level Structure

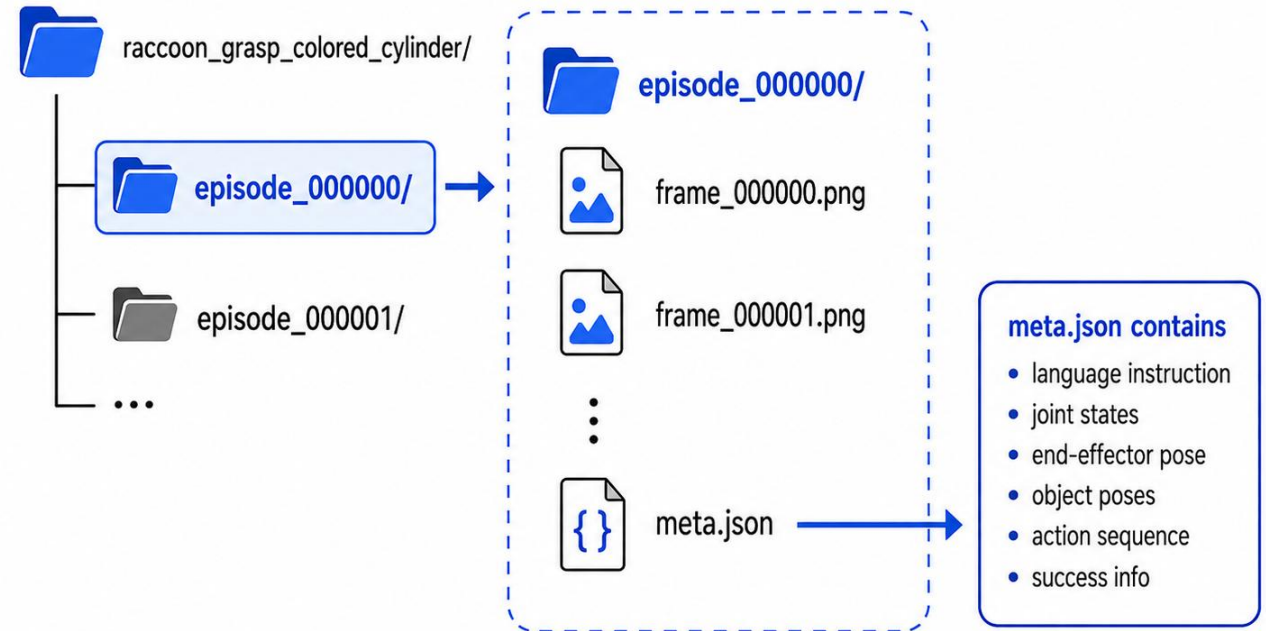
- Information stored in each episode

❖ Each episode contains

- Camera frames
- Language instruction
- Joint states
- End-effector pose
- Object poses
- Action sequence
- Success information

❖ Key Point

- Each episode stores both visual observations and robot-control information.



Step 1: Raw Episode Generation

❖ Commands

- `cd /data/Raccoonbot_Openvla/Mujoco`
- `python raccoon_grasp_multicolor_scene_dataset.py`

Generating 400 episodes may take about 2 hours.
Reduce the number of episodes for a quick practice run.

❖ Results

The screenshot displays two file explorer windows. The left window shows the directory `/ ... / Mujoco / raccoon_grasp_colored_cylinder /` with a table of files:

Name	Last Modified
episode_000001	10 minutes ago
episode_000002	10 minutes ago
episode_000003	10 minutes ago
episode_000004	10 minutes ago
episode_000005	10 minutes ago
episode_000006	9 minutes ago
episode_000007	9 minutes ago
episode_000008	9 minutes ago
episode_000009	9 minutes ago
episode_000010	9 minutes ago

The right window shows the directory `/ ... / raccoon_grasp_colored_cylinder / episode_000001 /` with a table of files:

Name	Last Modified
frame_000000.png	11 minutes ago
frame_000001.png	11 minutes ago
frame_000002.png	11 minutes ago
frame_000003.png	11 minutes ago
frame_000004.png	11 minutes ago
frame_000005.png	11 minutes ago
frame_000006.png	11 minutes ago
frame_000007.png	11 minutes ago
frame_000008.png	11 minutes ago
frame_000009.png	11 minutes ago
frame_000010.png	11 minutes ago

A pink arrow points from the `episode_000001` folder in the left window to the `episode_000001` folder in the right window.

Step 2: Convert Raw Data to RLDS Format

❖ Goal

- Convert raw MuJoCo episodes into an OpenVLA-compatible dataset format.

❖ Input

- Episode folders containing `frame_*.png` and `meta.json`.

❖ Main Script

- `convert_raw_to_openvla_rlds_intermediate.py`

❖ What the Conversion Does

- Reads each episode folder.
- Loads image frames and `meta.json`.
- Converts data into step-level records.
- Creates train / validation split.
- Prepares the dataset for the TFDS builder.

Step 2: Convert Raw Data to RLDS Format

❖ Commands

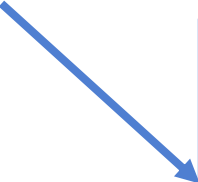
```
cd /data/Raccoonbot_Openvla/Mujoco/raccoon_dataset
```

실제 clone 경로 기준이면,
`cd /data/code/Raccoonbot_Openvla/Mujoco/raccoon_dataset`

```
python convert_raw_to_openvla_rlds_intermediate.py \  
--raw_root /data/Raccoonbot_Openvla/Mujoco/raccoon_grasp_colored_cylinder \  
--out_root /data/Raccoonbot_Openvla/Mujoco/raccoon_dataset/openvla_rlds_intermediate \  
--val_ratio 0.1
```

❖ Output

- `openvla_rlds_intermediate/`
- `dataset_info.json`
- `manifest_train.jsonl`
- `manifest_val.jsonl`
- `train/` and `val/` episode folders



```
[OK] val/episode_000272 | steps 25 -> 25 | success=True  
[OK] val/episode_000281 | steps 25 -> 25 | success=True  
[OK] val/episode_000293 | steps 25 -> 25 | success=True  
[OK] val/episode_000306 | steps 25 -> 25 | success=True  
[OK] val/episode_000339 | steps 25 -> 25 | success=True  
[OK] val/episode_000348 | steps 25 -> 25 | success=True  
[OK] val/episode_000355 | steps 25 -> 25 | success=True  
[OK] val/episode_000365 | steps 25 -> 25 | success=True  
[OK] val/episode_000368 | steps 25 -> 25 | success=True  
[OK] val/episode_000373 | steps 25 -> 25 | success=True  
  
Done.  
train episodes: 360  
val episodes : 40  
output root : /data/code/Raccoonbot_Openvla/Mujoco/raccoon_dataset/openvla_rlds_intermediate
```

This step bridges raw simulation logs and the dataset format required for OpenVLA fine-tuning.

Step 2: Build TFDS Dataset

❖ Goal

- Build a TensorFlow Dataset from the RLDS intermediate data.

❖ Why This Step Is Needed

- OpenVLA fine-tuning expects a dataset in TFDS / RLDS format.
- The raw intermediate files are not used directly for training.
- The dataset builder defines the final feature structure.

❖ Main Builder

- `Mujoco/rlds_dataset_builder/raccoon_pick_place/`
- `raccoon_pick_place_dataset_builder.py`

❖ Key Feature Shapes

- image: (256, 256, 3)
- state: (8,)
- action: (7,)

image: RGB observation
state: padded robot state
action: OpenVLA-style 7D action

Step 2: Build TFDS Dataset

❖ Commands

```
cd /data/Raccoonbot_Openvla/Mujoco/rlds_dataset_builder/raccoon_pick_place  
tfds build --overwrite
```

❖ Output

- tensorflow_datasets/raccoon_pick_place/



```
Dataset raccoon_pick_place downloaded and prepared to /root/tens  
calls will reuse this data.  
INFO[build.py]: Dataset generation complete...
```

❖ Move It to the Project Directory

- Remove existing tensorflow_datasets if needed.

```
✓ # rm -rf /data/Raccoonbot_Openvla/tensorflow_datasets
```

- mv /root/tensorflow_datasets /data/Raccoonbot_Openvla/

```
At notebook cell:  
%cd  
!mv
```

❖ Final Dataset Location

- /data/Raccoonbot_Openvla/tensorflow_datasets/

❖ Takeaway

- This step creates the final training dataset read by the OpenVLA fine-tuning script.

Step 2: TFDS Dataset Generated

❖ Dataset Summary

- Dataset: raccoon_pick_place/1.0.0
- Format: TFRecord
- Train: 360 examples
- Validation: 40 examples
- Total: 400 examples

❖ Output Files

- Metadata files: dataset_info.json, features.json
- Training split: 2 TFRecord shards
- Validation split: 1 TFRecord shard

❖ Takeaway

- The dataset is ready to be used by the OpenVLA fine-tuning script.



/ ... / raccoon_pick_place / 1.0.0 /	
Name	
{}	dataset_info.json
{}	features.json
📄	raccoon_pick_place-train.tfrecord-00000-of-00002
📄	raccoon_pick_place-train.tfrecord-00001-of-00002
📄	raccoon_pick_place-val.tfrecord-00000-of-00001

Raw Episode Visualization

❖ Example

- Instruction: "grasp the green cylinder"
- Target color: green
- Success: True

❖ Visual Check

- Consecutive frames show the robot moving toward the target object.
- Metadata confirms the language instruction and task result.



Optional: Raw Episode Visualization Code

```
import json
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path

root = Path("/data/Raccoonbot_Openvla/Mujoco/raccoon_grasp_colored_cylinder")
ep = sorted([p for p in root.iterdir() if p.is_dir() and p.name.startswith("episode_"))][0]

frames = sorted(ep.glob("frame_*.png"))
with open(ep / "meta.json", "r") as f:
    meta = json.load(f)

instruction = meta.get("instruction", meta.get("language_instruction", "N/A"))
target_color = meta.get("target_color", "N/A")
success = meta.get("success", "N/A")
n_show = min(5, len(frames))
fig, axes = plt.subplots(1, n_show, figsize=(3*n_show, 3))

for ax, frame_path in zip(axes, frames[:n_show]):
    img = Image.open(frame_path)
    ax.imshow(img)
    ax.set_title(frame_path.name, fontsize=10)
    ax.axis("off")

plt.suptitle(
    f"Instruction: {instruction}\nTarget Color: {target_color}, Success: {success}",
    fontsize=12
)
plt.tight_layout()
plt.show()
```

OpenVLA Fine-Tuning

❖ Goal

- Adapt `openvla/openvla-7b` to the RaccoonBot grasping task.

❖ Input Dataset

- `raccoon_pick_place`
- TFDS root: `/data/Raccoonbot_Openvla/tensorflow_datasets/`

❖ Fine-Tuning Method

- LoRA fine-tuning
- Base model parameters are mostly frozen.
- Lightweight adapter parameters are trained.

❖ Main Idea

- OpenVLA foundation model + RaccoonBot TFDS dataset → adapted VLA policy

Step 3: OpenVLA LoRA Fine-Tuning

❖ Main Script

- `openvla/vla-scripts/finetune.py`

❖ Commands

```
cd /data/Raccoonbot_Openvla/openvla
export PYTHONPATH=/data/Raccoonbot_Openvla/openvla:$PYTHONPATH
# %env PYTHONPATH=/data/Raccoonbot_Openvla/openvla # notebook cell

WANDB_MODE=disabled CUDA_VISIBLE_DEVICES=0 \
torchrun --standalone --nnodes 1 --nproc-per-node 1 vla-scripts/finetune.py \
  --vla_path openvla/openvla-7b \
  --data_root_dir /data/Raccoonbot_Openvla/tensorflow_datasets \
  --dataset_name raccoon_pick_place \
  --run_root_dir /data/Raccoonbot_Openvla/openvla/openvla-runs \
  --adapter_tmp_dir /data/Raccoonbot_Openvla/openvla/openvla-adapter-tmp \
  --lora_rank 32 \
  --batch_size 8 \
  --grad_accumulation_steps 2 \
  --learning_rate 5e-4 \
  --max_steps 30000 \
  --save_steps 30000 \
  --run_id_note raccoon-eef-v100
```

Effective batch size: 16
Training samples: 360
≈ 360/16 (22.5) steps per epoch

For a quick practice run:

- `max_steps = 100`
- `save_steps = 100`

Fine-Tuning Configuration

❖ Model and Dataset

- Base model: `openvla/openvla-7b`
- Dataset: `raccoon_pick_place`

❖ Training Method

- LoRA fine-tuning
- LoRA rank: 32

❖ Training Setup

- GPU: 1 × A100
- Practice run: 100 steps
 - ✓ 100 steps is for checking the pipeline, not for final performance.
- Full run: 30,000 steps

A full 30,000-step run may take 12+ hours on a single A100.

❖ Takeaway

- We fine-tune LoRA adapters, not the full 7B model.

MuJoCo Inference

MuJoCo Inference

❖ Goal

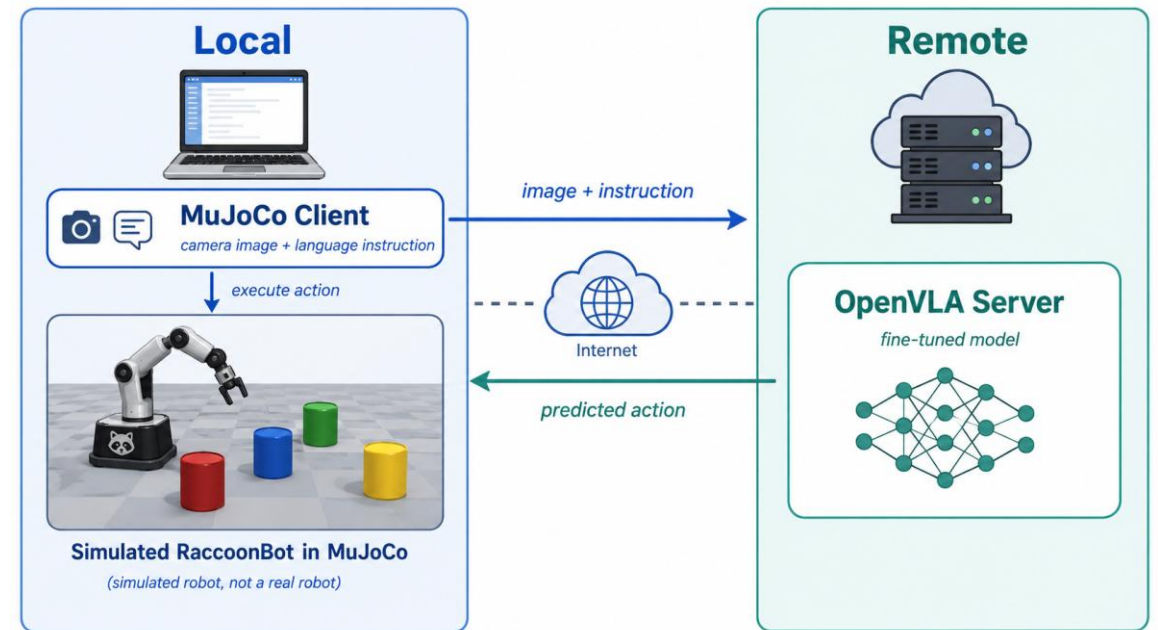
- Run OpenVLA as an inference server.
- Use MuJoCo as the client-side robot environment.
- Execute predicted actions in simulation.

❖ Inference Flow

- Local: MuJoCo Client
 - ✓ Captures image and instruction.
 - ✓ Executes predicted action.
- Remote: OpenVLA Server
 - ✓ Receives image + instruction.
 - ✓ Sends action back to the client

❖ Key Point

- This example controls RaccoonBot in MuJoCo simulation, not a physical robot.



Option A: Use Hugging Face Checkpoint

❖ Purpose

- Use a pre-finetuned RaccoonBot OpenVLA model.
- Skip local dataset generation and fine-tuning.
- Run MuJoCo inference directly.

❖ Download

```
pip install -U huggingface_hub  
  
hf download fair-lab/openvla-7b-finetuned-raccoonbot \  
  --local-dir /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b-finetuned-raccoonbot
```

```
model-00001-of-00004.safetensors: 100%|██████████| 4.93G/4.93G [23:48<00:00, 3.45MB/s]  
Download complete. Moving file to /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b-finetuned-raccoonbot/mode  
l-00001-of-00004.safetensors  
Fetching 15 files: 100%|████████████████████████████████████████| 15/15 [23:48<00:00, 95.27s/it]  
/data/code/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b-finetuned-raccoonbot
```

- Download may take 30+ minutes.

❖ Option A: Use Hugging Face Checkpoint

```
cd /data/Raccoonbot_Openvla/openvla

CUDA_VISIBLE_DEVICES=0 python openvla_server.py \
  --model_path /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b-finetuned-raccoonbot \
  --default-unnorm-key raccoon_pick_place \
  --host 0.0.0.0 \
  --port 8000 \
  --device cuda
```

❖ Option B: Use Our Fine-Tuned Checkpoint

```
cd /data/Raccoonbot_Openvla/openvla

CUDA_VISIBLE_DEVICES=0 python openvla_server.py \
  --model_path /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b+raccoon_pick_place+b16+lr-0.0005+lora-r32+dropout-0.0--raccoon-eef-v100--image_aug \
  --default-unnorm-key raccoon_pick_place \
  --host 0.0.0.0 \
  --port 8000 \
  --device cuda
```

- `raccoon\_pick\_place` selects the action normalization statistics for RaccoonBot.

Step 4: Client-Side Environment Setup

❖ Goal

- Set up the local MuJoCo client.

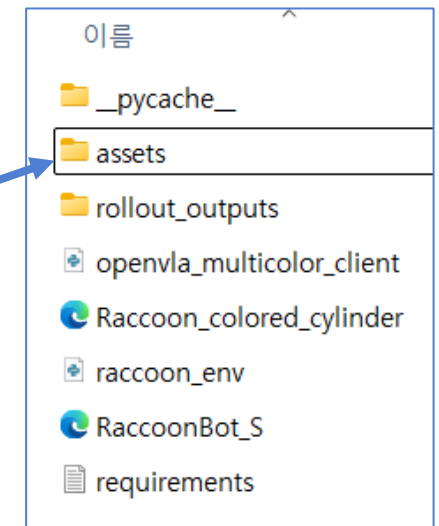
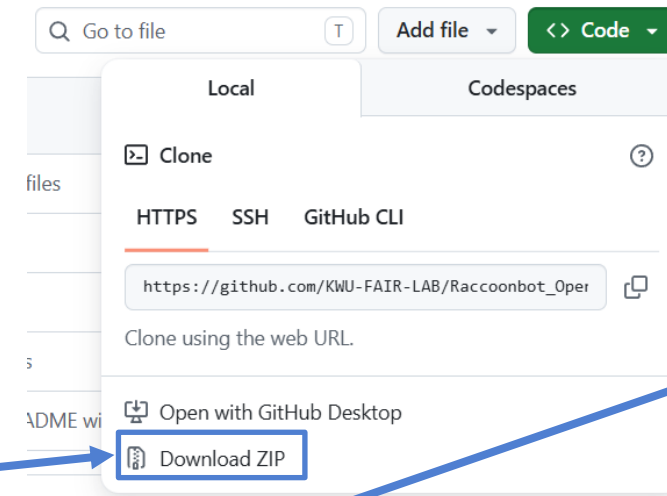
❖ Required Client Files

- openvla_multicolor_client.py
- raccoon_env.py
- Raccoon_colored_cylinder.xml
- RaccoonBot_S.xml
- requirements.txt
- assets/

Download client files

❖ Important: Copy assets

- Download the GitHub ZIP.
- Copy Mujoco/assets/ into the local client folder.
- Confirm that assets/Link2.STL exists.



Step 4: Install

❖ Install Requirements

```
cd "D:\Choi_Lecture\Spring_2026\피지컬AI\라쿤봇\ mujoco_openvla_client"  
pip install -r requirements.txt
```

```
Requirement already satisfied: invoke>=2.0 in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from  
=2.7.2->sshtunnel->-r requirements.txt (line 5)) (3.0.3)  
Requirement already satisfied: pynacl>=1.5 in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from  
=2.7.2->sshtunnel->-r requirements.txt (line 5)) (1.6.2)  
Requirement already satisfied: cffi>=2.0.0 in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from  
phy>=3.3->paramiko>=2.7.2->sshtunnel->-r requirements.txt (line 5)) (2.0.0)  
Requirement already satisfied: pycparser in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from  
->cryptography>=3.3->paramiko>=2.7.2->sshtunnel->-r requirements.txt (line 5)) (3.0)  
Requirement already satisfied: fsspec in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from e  
mujoco->-r requirements.txt (line 1)) (2026.4.0)  
Requirement already satisfied: typing_extensions in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packa  
ls[epath]->mujoco->-r requirements.txt (line 1)) (4.15.0)  
Requirement already satisfied: zipp in C:\Users\User-Choi\AppData\Local\Programs\Python\Python313\Lib\site-packages (from eti  
joco->-r requirements.txt (line 1)) (4.1.0)
```

Step 4: Connect via SSH Tunnel

❖ Goal

- Connect the local MuJoCo client to the remote OpenVLA server.
- Use SSH tunneling when server port 8000 is not directly accessible.
- Keep the tunnel open while running the client.

❖ Open SSH Tunnel

- `ssh -L 8000:127.0.0.1:8000 root@qlak315.iptime.org -p 23000`

❖ Key Point

- Do not close this terminal window.
- After tunneling, the client can use `http://127.0.0.1:8000`.



The client connects to a local address, but the request is served remotely.

Step 4: Run MuJoCo Client

❖ Check Connection

- `curl http://127.0.0.1:8000`

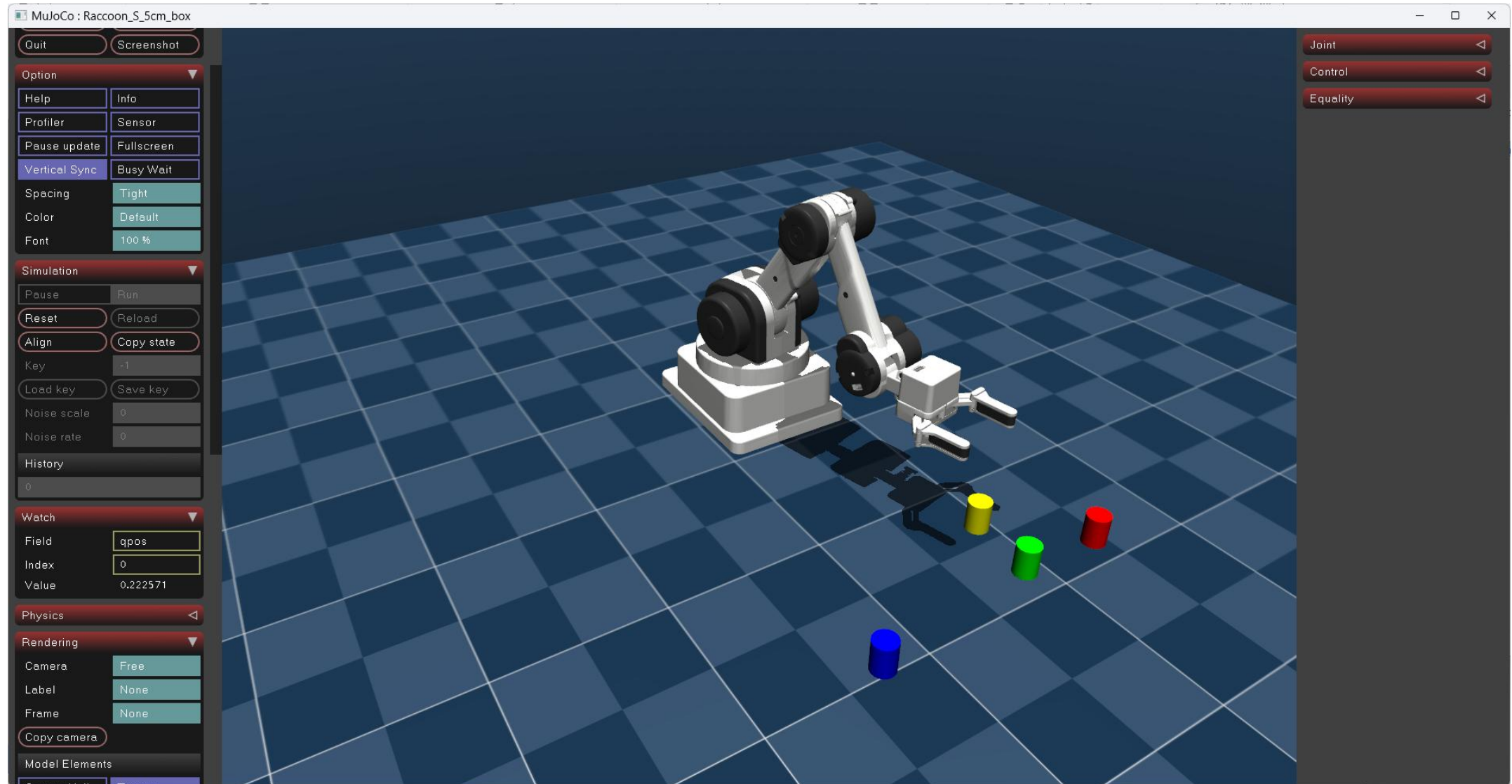
❖ Expected Response

- `{"detail": "Not Found"}`

❖ Run Client

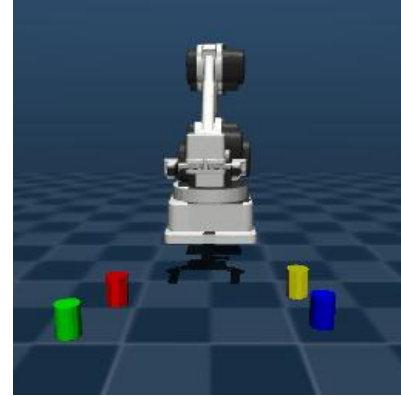
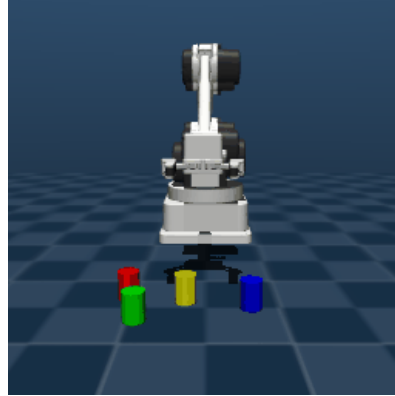
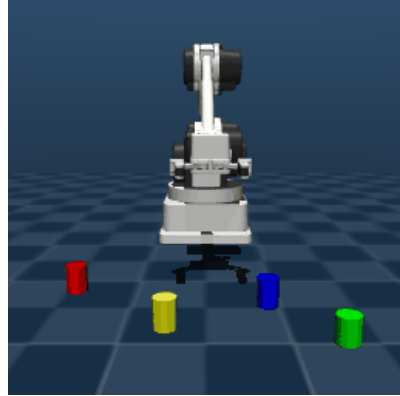
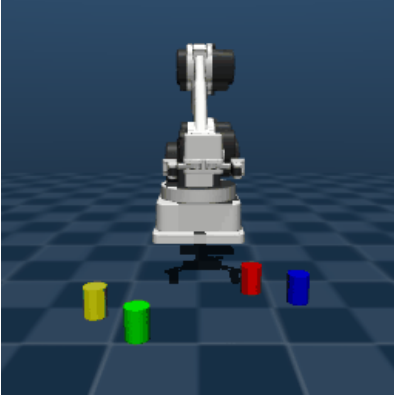
- `cd "D:\Choi_Lecture\Spring_2026\피지컬AI\라쿤봇\mujoco_openvla_client"`
- `python openvla_multicolor_client.py --server_url http://127.0.0.1:8000 --xml_path Raccoon_colored_cylinder.xml --target_color red --use_viewer`

MuJoCo Client Running

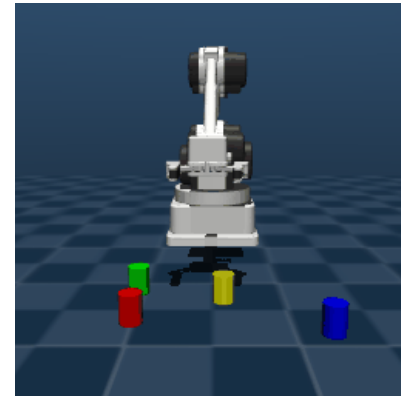
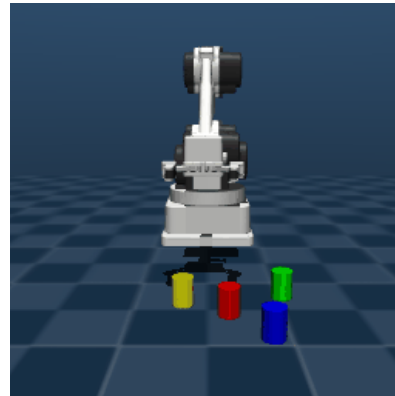
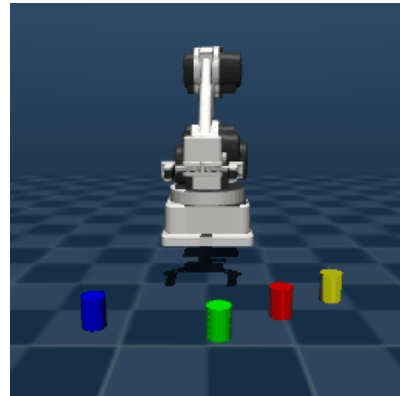
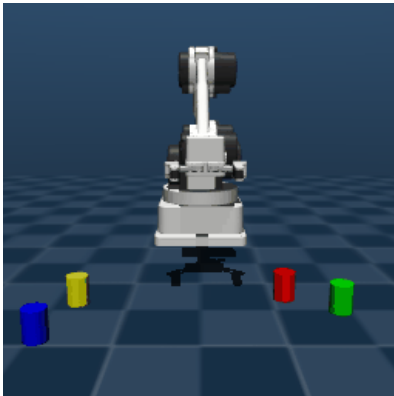


Checkpoint Comparison Test

❖ 1,000 Steps



❖ 30,000 Steps



Prompt: grasp the [red/blue/yellow/green] cylinder

Assignment 1: Extending the RaccoonBot OpenVLA Pipeline

1. Dataset Extension

❖ Current Limitation

- Object type: cylinder only
- Task type: grasp only
- Single instruction template: "grasp the {color} cylinder"

❖ Choose Multiple Extensions

- Add **new objects**: cube, sphere, box, etc.
- Add **new tasks**: push, lift, pick-and-place, etc.
- Add more diverse language instructions.

❖ Required Results

- Generate new MuJoCo demonstrations
- Rebuild RLDS / TFDS dataset
- Visualize one episode and run a short LoRA test

2. Code Improvement

❖ Choose at Least Two

- Improve 7D-to-4DOF action mapping.
- Speed up inference or motion execution.
- Add timing/action logs or better visualization.

❖ Bonus

- Try **physical RaccoonBot control**, if available.

❖ Required Results

- Implement one meaningful change.
- Show before / after evidence.
- Discuss the effect on the VLA pipeline.

❖ Key Question

- What makes the system faster, clearer, or more reliable?

❖ Assignment Submission

- Submit a GitHub repository URL by June 7, 2026, 11:59 PM.
- Include modified code, README.md, logs, screenshots, and a short report.
- Do not upload large datasets or model checkpoints.
- Optional: include a short demo video.

❖ Repository Should Include

- README.md
- Modified source files
- Result screenshots or episode visualizations
- Training / inference logs
- report.pdf

❖ Key Rule

- Show what you changed, how to run it, and what result you obtained.